# GitHUB Repository link

https://github.com/KevinHonningsvaag/deliverable_2_inf201

# Deliverable 2

Authors: Kevin Honningsvåg & Herman Nordstrand

## Imports

In [65]:
```python
import json
import re
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
```

## Task 1

In [66]:
```python
def read_log_file(file_path: Path) -> tuple[str, str]:
    """Read a log file and return its content and encoding."""
    try:
        # Try to read the file with UTF-8
        with open(file_path, "r", encoding="utf-8") as file:
            content = file.read()
        return content, "utf-8"
    except UnicodeDecodeError:
        # If decoding with UTF-8 fails, try Latin-1
        with open(file_path, "r", encoding="latin-1") as file:
            content = file.read()
        return content, "latin-1"


def parse_log(text: str) -> list[dict[str, str | float]]:
    """Extract valid transactions from log text."""
    pattern = (
        r"\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]"
        r" \[(?P<register_id>POS_\d+)\]"
        r" item=(?P<item>\w+)"
        r" price=(?P<price>\d+\.\d{2})"
        r" (?P<currency>EUR|NOK|USD)"
        r" \((?P<status>SUCCESS|REFUND|CANCELLED)\)"
    )

    transactions = []

    for line in text.splitlines():
        match = re.fullmatch(pattern, line.strip())

        if match:
```

```
            transaction = {
                "timestamp": match.group("timestamp"),
                "register_id": match.group("register_id"),
                "item": match.group("item"),
                "price": float(match.group("price")),
                "currency": match.group("currency"),
                "status": match.group("status"),
            }

            transactions.append(transaction)

    return transactions
```

# Task 2

In [67]:
```
def find_log_files(folder: Path) -> list[Path]:
    """Recursively find all .log files in a directory."""
    folder = Path(folder)
    log_files: list[Path] = []

    for item in folder.iterdir():
        if item.is_dir():
            log_files.extend(find_log_files(item))

        elif item.is_file() and item.suffix == ".log":
            log_files.append(item)

    return log_files


def find_active_registers(data: Any) -> set[str]:
    """Recursively find all unique register IDs from the parsed log data."""
    active_registers: set[str] = set()
    if isinstance(data, dict):
        if data.get("active") is True:
            register_id = data.get("id")

            if isinstance(register_id, str):
                active_registers.add(register_id)

        for value in data.values():
            active_registers.update(find_active_registers(value))

    elif isinstance(data, list):
        for item in data:
            active_registers.update(find_active_registers(item))

    return active_registers
```

# Task 3

In [68]:
```
def compute_refund_ratio(
    sales_count: np.ndarray,
    refund_count: np.ndarray,
) -> np.ndarray:
    """Calculate the ratio off how often a refund ocuorce"""
```

```
    epsilon: float = 1e-6
    return refund_count / (sales_count + epsilon)


def remove_price_outliers(price_data: np.ndarray) -> np.ndarray:
    """Clean prices by replacing invalid prices with a median of the
    valid ones"""
    prices = np.asarray(price_data, dtype=float).reshape(-1)

    # Controls that the array isnt empty
    if prices.size == 0:
        return prices

    mean_price = np.mean(prices)
    std_price = np.std(prices)

    # Controls that all the the prices isnt the same
    if std_price == 0:
        return prices.copy()

    outlier_mask = np.abs(prices - mean_price) > 3 * std_price
    valid_prices = prices[~outlier_mask]

    # Controls that there is valid prices
    if valid_prices.size == 0:
        return prices.copy()

    median_price = np.median(valid_prices)

    cleaned_prices = prices.copy()
    cleaned_prices[outlier_mask] = median_price

    return cleaned_prices
```

# Task 4

In [69]:

```
def make_dataframe(
    transactions: list[dict[str, Any]],
) -> pd.DataFrame:
    """Convert a list of transaction dictionaries to a pandas DataFrame."""
    df = pd.DataFrame(transactions)
    df['timestamp'] = pd.to_datetime(
        df['timestamp'],
        format="%Y-%m-%d %H:%M:%S",
    )
    df = df.set_index('timestamp')
    df = df.sort_index()

    return df


def hourly_sales_per_item(df: pd.DataFrame) -> pd.DataFrame:
    """Takes the DataFrame groups it by item and returns the sum of sales
    for the following hour"""
    grouped = df.groupby('item')
    hourly_sales = grouped.resample("1h")

    result = hourly_sales['price'].sum()
```

```
    # makes result in to a dataFrame
    result = result.reset_index()
    return result


def create_pivot_table(df: pd.DataFrame) -> pd.DataFrame:
    """Creates a pivot table using the dataframe"""

    pivot = df.pivot_table(
        index='item',
        columns='status',
        values='price',
        aggfunc="sum",
        # To get 0 instead of NaN if there is any
        fill_value=0,
    )
    pivot = pivot.reindex(
        columns=['SUCCESS', 'REFUND', 'CANCELLED'],
        fill_value=0,
    )

    return pivot


def refund_ratio_per_item(df: pd.DataFrame) -> pd.DataFrame:
    """Calculate the refund ratio for each item."""

    counts = df.groupby(['item', 'status']).size()
    counts = counts.unstack(fill_value=0)
    counts = counts.reindex(
        columns=['SUCCESS', 'REFUND', 'CANCELLED'],
        fill_value=0,
    )
    sales_count = counts['SUCCESS'].to_numpy()
    refund_count = counts['REFUND'].to_numpy()
    ratios = compute_refund_ratio(sales_count, refund_count)
    counts['refund_ratio'] = ratios

    return counts
```

# Task 5

In [70]:

```
def main() -> None:
    #base_dir = Path(__file__).resolve().parent does not work in Jupyter noteboo
    # so we have to replace it for the code to work
    base_dir = Path.cwd()
    data_dir = base_dir / "data"
    config_path = data_dir / "store_config.json"

    log_files = find_log_files(data_dir)

    with open(config_path, "r", encoding="utf-8") as file:
        config = json.load(file)

    active_registers = find_active_registers(config)

    transactions: list[dict[str, Any]] = []
```

```
    for log_file in log_files:
        content, encoding = read_log_file(log_file)
        parsed_transactions = parse_log(content)
        transactions.extend(parsed_transactions)

        print(f"Read {log_file.name} using {encoding}:")
        print(f"{len(parsed_transactions)} transactions")

    transactions = [
        transaction
        for transaction in transactions
        if transaction["register_id"] in active_registers
    ]

    df = make_dataframe(transactions)

    df['price'] = remove_price_outliers(df['price'].to_numpy())

    nok_mask = df['currency'] == "NOK"
    df.loc[nok_mask, 'price'] *= 10.0

    hourly_revenue = hourly_sales_per_item(df)
    hourly_revenue['price'] = hourly_revenue['price'].round(2)
    status_pivot = create_pivot_table(df).round(2)
    refund_ratios = refund_ratio_per_item(df)

    item_refund_ratios = refund_ratios[['refund_ratio']]

    df.to_csv(base_dir / "cleaned_transactions.csv")

    hourly_revenue.to_json(
        base_dir / "hourly_revenue.json",
        orient="records",
        date_format="iso",
        indent=2,
    )
    status_pivot.to_csv(base_dir / "status_pivot_table.csv")
    item_refund_ratios.to_csv(base_dir / "item_refund_ratios.csv")

    print("\nWorkflow completed")
    print(f"Log files found: {len(log_files)}")
    print(f"Active registers: {sorted(active_registers)}")
    print(f"Transactions processed: {len(df)}")

#Show that we have done the task in a presentable way
#So it is easy to see if something is wrong
    print("\n========== Display ==========")

    # Log files and registers
    print(f"\nLog files found: {len(log_files)}")
    print(f"Active registers: {sorted(active_registers)}")
    print(f"Transactions processed: {len(df)}")

    # Cleaned transaction data
    print("\n========== CLEANED TRANSACTIONS ==========")
    print(df.head())
    # Currency correction
    print("\n========== CURRENCY CORRECTION ==========")
    print(f"NOK transactions corrected: {nok_mask.sum()}")
```

```
    # Hourly revenue per item
    print("\n========== HOURLY REVENUE ==========")
    print(hourly_revenue.head(10))

    # Pivot table
    print("\n========== STATUS PIVOT TABLE ==========")
    print(status_pivot)

    # Refund ratios
    print("\n========== REFUND RATIOS PER ITEM ==========")
    print(refund_ratios)

    # Output files
    print("\n========== OUTPUT FILES ==========")

    for filename in [
        "cleaned_transactions.csv",
        "hourly_revenue.json",
        "status_pivot_table.csv",
        "item_refund_ratios.csv",
    ]:
        print(f"{filename}: {(base_dir / filename).exists()}")


if __name__ == "__main__":
    main()
```

```
Read register_sales.log using utf-8:
80 transactions
Read register_sales.log using utf-8:
80 transactions
Read register_sales.log using utf-8:
80 transactions
Read register_sales.log using utf-8:
80 transactions
Read register_sales.log using utf-8:
80 transactions
Read register_sales.log using utf-8:
80 transactions
Read register_sales.log using utf-8:
80 transactions
Read register_sales.log using utf-8:
80 transactions
Read register_sales.log using utf-8:
80 transactions
Read register_sales.log using utf-8:
80 transactions

Workflow completed
Log files found: 9
Active registers: ['POS_01', 'POS_02', 'POS_03']
Transactions processed: 720

========== Display ==========

Log files found: 9
Active registers: ['POS_01', 'POS_02', 'POS_03']
Transactions processed: 720

========== CLEANED TRANSACTIONS ==========
                    register_id       item  price currency   status
timestamp
2026-10-15 07:00:30      POS_03  Croissant   3.58      EUR  SUCCESS
2026-10-15 07:00:36      POS_02   Espresso   3.21      EUR  SUCCESS
2026-10-15 07:00:49      POS_03   Espresso  25.20      NOK  SUCCESS
2026-10-15 07:01:01      POS_01   Espresso   2.57      EUR  SUCCESS
2026-10-15 07:01:12      POS_03   Sandwich   7.78      EUR  SUCCESS

========== CURRENCY CORRECTION ==========
NOK transactions corrected: 227

========== HOURLY REVENUE ==========
         item           timestamp    price
0  Cappuccino 2026-10-15 07:00:00   734.54
1  Cappuccino 2026-10-15 08:00:00   663.57
2  Cappuccino 2026-10-15 09:00:00    61.15
3   Croissant 2026-10-15 07:00:00  1024.91
4   Croissant 2026-10-15 08:00:00   498.41
5   Croissant 2026-10-15 09:00:00   125.66
6    Espresso 2026-10-15 07:00:00   593.67
7    Espresso 2026-10-15 08:00:00   754.38
8    Espresso 2026-10-15 09:00:00    46.50
9       Latte 2026-10-15 07:00:00   627.04

========== STATUS PIVOT TABLE ==========
status      SUCCESS  REFUND  CANCELLED
item
Cappuccino   994.50  192.26     272.50
Croissant    915.21  474.29     259.48
```

```
Espresso     930.35  175.76     288.44
Latte       1336.29  378.28     231.24
Muffin      1547.86  196.31     447.95
Sandwich    2568.35  540.58     403.95

========== REFUND RATIOS PER ITEM ==========
status      SUCCESS  REFUND  CANCELLED  refund_ratio
item
Cappuccino       69      16         20      0.231884
Croissant        71      28         24      0.394366
Espresso         76      18         24      0.236842
Latte            89      24         16      0.269663
Muffin           84      14         25      0.166667
Sandwich         87      17         18      0.195402

========== OUTPUT FILES ==========
cleaned_transactions.csv: True
hourly_revenue.json: True
status_pivot_table.csv: True
item_refund_ratios.csv: True
```

# Discussion of results and code

We feel like we got the desired result, though not quite even distribution of work throughout the two weeks as there have been some sickness and flu. But we got there in the end anyway. But in this assignment, we built a prgram to read, clean and analyse fictional transaction data from multiple coffee shops on campus. When writing the script we followed the workflow of the tasks and put it together at the end.

in Task 1, we used regex (regular expressions) to extract transactional information from the log files. We used try and except to read different types of encoding, so it tries with utf-8 and if that fails it tries with latin-1. Though the result only shows UTF-8. That might be because the characters in the files is able to be read by both encodings, so we dont know which one it originally was.

I Task 2 we used recursion to locate log files in nested folders and identify the active registers from the JSON configuration file. This makes it so that the program can handle different folder structures without having to specify the path manually in the script.

For task 3 we used NumPy as a way to detect and replace outliers, so that prices more than three standard deviations form the mean would be replaced with the median of the remaining values. This prevents some extreme prices such as 999.00 which was a problem before we started using NumPy. There are some issues with this method though, as extreme values can affect the mean and standard deviation used to detect them. So you have to take it with a grain of salt for this one, but it is still better than no fix.

Task 4 we used Pandas to organise the transactions chronologically and then we would further organise versions of this with grouping, reampling and pivot tables. This allowed us to see the transactional values by item, hour and status. We also calculated the refund ratios by counting successful sales and refunds for each item and used the formula.

For Task 5 we bascially put all the other tasks together.

As for the results, we can see the program found 9 log files, identified 3 acitve registers and processed 720 transactions. And generated the required CSV and JSON reports.

The currency are summed without converting them into a common one, which means that the reported totals might be off based on exchange rate, so it would be wrong to interpret the data as actual revenue in one currency.

## GenAI

We have tried to use as little GenAi as possible, as we want to learn and do it mostly by ourselves. But we have used some AI to help bugfixing, and if we have done the task the right way, as a way to double check if we interpereted the task in the right way. So it has been sort of a guy in your class who knows more than you, that you ask when your stuck.

## Sources or pages with useful info

https://regexr.com/

https://stackoverflow.com/questions/69049818/how-to-export-jupyter-notebook-by-vscode-in-pdf-format-windows-10